Shaping Gap Benchmark: Effect Interfaces for Compositional Execution of Frozen Neural Computation

Nathaniel Oh Paul Attie

November 12, 2026

Abstract

We introduce Shaping Gap Benchmark and a reference runtime for acquiring reusable behavioral interfaces over native components of an ordinary frozen pretrained model. The runtime uses those interfaces to construct computations satisfying explicit neural requirements. An interface describes a component's accumulated effect and how incoming context changes that effect. Integrating this context sensitivity along a predecessor's displacement gives a compatibility forecast for composition. The acquired descriptions persist across requirements; the runtime constructs and deploys temporary native programs from them. We implement source acquisition, bounded retained arrays, compatibility-guided branching execution, and independent deployment. A controlled study compares derivative-supervised quadratic acquisition, endpoint-only acquisition, and a structure-calibrated reference across three local activation domains and 288 fresh numerical contracts. The equal-context endpoint interface satisfies 100 contracts versus 80 for local native sensitivity guidance, with 15.76% less acquisition-and-execution native-query work. Under the same online envelope, the differential interface satisfies 104, compared with 105 for query-matched endpoint acquisition. Its context-partial prediction improves in all three domains; conditioning removal and permutation satisfy 105 and 108, respectively, locating an unresolved gap between predictive fidelity and construction benefit. The structure-calibrated reference satisfies 129 with 21.39% less source-and-execution native-query work than local guidance. All 2592 selected programs replay exactly through original native components, with pretrained parameters unchanged. A standalone structured representation additionally reproduces all selected programs' contract decisions without loading the checkpoint, with maximum readout error 3.31×10^{-6}. The contribution is an acquired behavioral interface used as a construction abstraction, together with a benchmark that measures its construction value after acquisition and execution costs.

1 Introduction

A pretrained network supplies a large collection of learned computations. Its original architecture gives one way to execute them. Recent work on chain-of-layers and program-of-layers inference shows that frozen pretrained layers can also be skipped, repeated, and recomposed into input-specific programs [10, 11]. This makes an additional question concrete: *can reusable descriptions acquired from existing components improve the cost of constructing computations that satisfy later requirements?*

We study *effect-interface acquisition*. The runtime observes native components, retains descriptions of their behavior, and uses those descriptions to construct a computation satisfying an explicit neural contract. The contract specifies which monitored responses should change and which should

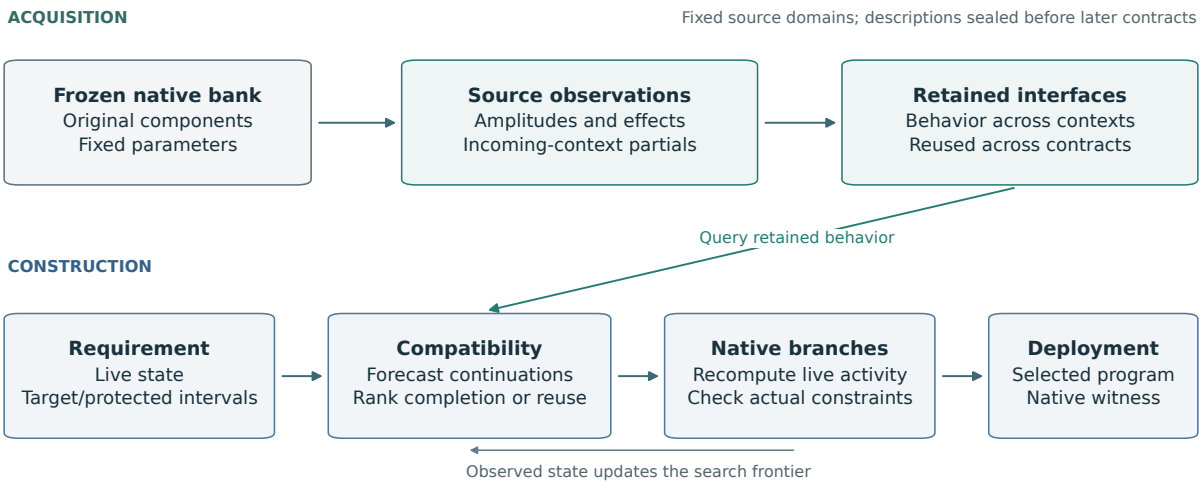


Figure 1: The acquired interface connects frozen computational material to requirement-guided construction. Source observations produce retained descriptions before the later contracts are sampled. At runtime the live state and contract query those descriptions; original native execution determines the result. The interface persists across contracts, while the search frontier and selected program belong to an individual construction.

remain within protected intervals. A constructed program is a sequence of original components acting on live activation. Its execution is checked through the original frozen parameters.

The key representation must describe more than a component in isolation. Executing A changes the state in which B operates, including B's own neural activity. A's immediate effect can therefore conceal a useful continuation, or make an apparently useful B ineffective. Our interface pairs B's accumulated effect E_B with its incoming-context derivative $K_B = DE_B$. Integrating K_B along A's displacement predicts how A conditions B. This turns compositional compatibility into a measurable, context-dependent quantity.

The interface is the retained object. Its coefficients describe behavior across source contexts and are reused under later contracts. The runtime uses them to forecast alternatives, classify compatible continuations, execute proposed native components, and deploy a selected program (Figure 1). Acquisition and construction are consequently separate stages with separate costs. In the present study, component boundaries, domains and monitored readouts are supplied; the acquired content is their numerical behavior.

Shaping Gap Benchmark makes this question reproducible. It holds the frozen component bank, requirements, native checker and online execution allowance fixed, varies the source knowledge used by the construction engine, and charges that knowledge's acquisition. The measured gap is a difference in achievable contract satisfaction or cost over the same computational material. This first study establishes the interface, its executable use, and its finite construction frontier; the research program is to make acquisition itself a continuing source of better constructions.

The study evaluates this separation directly. On 288 fresh numerical contracts, a 32-context endpoint interface satisfies 100 versus 80 for online native local guidance, with 15.76% less total native-query work after acquisition. A reference supplied with the native architectural form reaches 129 with 21.39% less work. Derivative supervision improves incoming-context prediction, but its extra conditioning term gives no net solve-count gain here. The controls separate predictive fidelity, construction value and acquisition return.

Our contributions are:

1. An accumulated-effect/context interface that connects finite neural behavior to predecessor-conditioned composition, with conditional prediction and decision-stability bounds.
2. A runtime that acquires and retains those descriptions, then uses them for compatibility-guided construction and deployment of original frozen components outside their original layer schedule.
3. Shaping Gap Benchmark: an acquisition-inclusive study with matched-engine controls, conditioning interventions, complete native witnesses, and standalone replay of the finite structure-calibrated bank.

The unit of retained learning is the behavioral interface; the unit of execution is a temporary native program; the evaluation criterion is construction quality against total charged work. This separation fixes the scope of the present paper and the questions advanced by follow-on acquisition studies.

2 Related work

Programs over frozen pretrained layers. CoLa searches for input-specific chains of pretrained layers using skipping, repetition and reordering [10]. PoLar subsequently learns a predictor of layer-segment programs from valid programs collected by MCTS [11]. These results establish alternative execution programs as a practical use of frozen pretraining. Our runtime acquires component behavior models and queries them under an explicit contract to forecast compositions. The studied components are native SwiGLU atoms with live input-dependent activities. Thus our organizing object is a reusable effect interface, while PoLar's predictor learns a direct map from input to execution program.

Decomposition, recomposition and representation alignment. Kingetsu et al. learn supermasks that extract functional modules from a trained classifier and recompose class subsets through mask unions and confidence voting [8]. Model stitching and latent functional maps connect representations from existing networks [1, 5]. Deja Vu predicts contextual sparsity [12]; automatic circuit discovery identifies influential subgraphs [2]. These methods provide precedents for exposing and reusing pretrained structure. We acquire predictive effects of supplied native atoms and compose their state transformations to satisfy new readout intervals. Table 1 identifies the different retained objects and construction decisions.

Neural program and skill libraries. Routing Networks, Neural Interpreters and Neural Attentive Circuits learn modular execution [15, 16, 22]. HOUDINI uses types and combinators to synthesize neural programs while learning their parameters [21]. DreamCoder and LILO acquire reusable program abstractions [4, 6]. Learned skill interfaces support planning over continuous behavior [9, 18]. We build on the idea that a reusable description can make a component accessible to a planner. Here the components come from an ordinary frozen pretrained model; the retained description specifies their numerical effects and context sensitivity.

Model-based neural control. Activation-LQR uses layerwise local dynamics and reusable offline-computed control gains to steer activations during inference [19]. It supplies a close

Method	Reusable object	Execution decision
PoLar [11]	Input-to-program predictor	Skip, keep or repeat layer segments
Module recomposition [8]	Learned class-subnetwork masks	Combine classifier modules
Activation-LQR [19]	Control gains from local neural dynamics	Steer activations during layerwise inference
HOUDINI [21]	Typed neural-function library	Synthesize and train a neural program
Our runtime	Native amplitude, accumulated effect and context partial	Forecast and construct a native program against a readout contract

Table 1: Methodological comparison. These works motivate the representation and execution design; the numerical study compares our common-engine controls, not performance on the cited works' application benchmarks.

precedent for amortizing knowledge of neural response. Our control variable is a discrete native-component program. Its effect forecast includes the successor's changed activity after a predecessor executes. The local sensitivity baseline tests whether such retained descriptions help beyond recomputing immediate native response at each search state.

Integrated derivatives and acquisition. Integrated Gradients accumulates response derivatives along paths; Integrated Hessians measures pairwise feature interactions [7, 20]. Sobolev training learns from function values and target derivatives [3]. These provide the mathematical and approximation building blocks. Our interface uses component execution and incoming state as its variables, and the runtime uses the resulting description to forecast a construction. The study therefore compares both prediction fidelity and its value in choosing executable programs. The structure-calibrated reference separately measures the value of supplying the known normalization and gating form.

3 The interface and its compositional meaning

3.1 Construction problem and benchmark objective

Let $\mathcal{B} = \{F_b\}$ be a supplied bank of frozen residual components, Ω an activation domain and q a supplied monitored readout. A later requirement consists of an initial state $x \in \Omega$ and an interval contract C on selected coordinates of q. A program $\pi = (b_1, \ldots, b_T)$ denotes $F_\pi = F_{b_T} \circ \cdots \circ F_{b_1}$. Its success condition is $q(F_\pi(x)) \in C$, with the benchmark's numerical tolerance. Program depth and the physical legality of every executed prefix are bounded by the declared protocol.

Acquisition observes native behavior and retains interface parameters θ. Later construction queries θ under each requirement and explores programs within an online native-query allowance. Native execution determines success. The benchmark records the retained array payload, execution workspace, native primitives and active time; each method's lifecycle includes its own source acquisition and subsequent construction and deployment.

For a fixed ordered stream, let $S_j(c)$ be the number of contracts satisfied by method j after cumulative charged work c, counting unattempted contracts as unsatisfied. The observed shaping gap between methods i, j is $S_i(c) - S_j(c)$; equivalently, one can compare work at the same attained count. Acquisition is charged at the curve's origin. Native-query work and active time give separate

curves, since neither substitutes for the other. This definition is an empirical construction frontier on the declared stream.

3.2 Accumulated effect and the partial of that effect

Let $x \in \mathbb{R}^d$ be live activation and $q : \mathbb{R}^d \to \mathbb{R}^m$ a fixed monitored neural readout. Component b is a compatible residual transformation $F_b(x) = x + f_b(x)$, with fixed pretrained parameters. Its total effect is

$$E_b(x) = q(F_b(x)) - q(x) = \int_0^1 J_q(x + sf_b(x))f_b(x)\, ds. \tag{1}$$

The identity follows by differentiating $q(x+sf_b(x))$ with respect to s and integrating coordinatewise. The path is an activation interpolation; s is not physical neural time or a trained weight.

Differentiating the accumulated effect with respect to incoming state gives

$$K_b(x) = D_x E_b(x) = J_q(F_b(x))D_x F_b(x) - J_q(x). \tag{2}$$

For a native atom $f_b(x) = D_b a_b(x)$, its complete derivative is $D_x F_b = I + D_b \nabla a_b(x)^\top$. Thus K_b includes the changed activity of the successor itself. Holding its amplitude constant would omit a term.

For predecessor displacement v, define $G(\alpha) = E_b(x + \alpha v)$. Then $G'(\alpha) = K_b(x + \alpha v)v$ and

$$E_b(x + v) - E_b(x) = \int_0^1 K_b(x + \alpha v)v\, d\alpha. \tag{3}$$

When $v = f_a(x)$, this is the finite interaction

$$I_{a\to b}(x) = q(F_b(F_a(x))) - q(F_a(x)) - q(F_b(x)) + q(x). \tag{4}$$

It differs from order dependence $q(F_b(F_a(x))) - q(F_a(F_b(x)))$. A nonlinear readout can yield non-additive effects even for commuting fixed writes.

Equivalently, with $Q(\alpha, \beta) = q(x+\alpha f_a(x)+\beta f_b(x+\alpha f_a(x)))$, the derivative of B's accumulated effect is $\partial_\alpha \int_0^1 \partial_\beta Q\, d\beta$. Under continuous mixed derivatives, integrating this quantity over α recovers the four-corner difference in Equation 4. This closes the chain from local partials, to accumulated effect, to the sensitivity of that accumulated effect.

Scope and trajectory information. The net effect in Equation 1 does not encode every intermediate violation or turn. An interface may additionally retain applicability, trajectory/protection evidence, uncertainty, and cost. The present central experiment executes live-input components; a separate fixed root source would require distinguishing incoming-state and root partials. We use real-arithmetic smoothness for the identities and numerical validation for float32 execution.

3.3 Conditional prediction and contract checking

Proposition 1 (Accumulated conditioning error)**.** *Suppose* $\|\widehat{E}_b(x) - E_b(x)\| \le \varepsilon_E$ *and* $\sup_{t\in[0,1]} \|\widehat{K}_b(x + tv) - K_b(x + tv)\|_{\mathrm{op}} \le \varepsilon_K$*. Then*

$$\left\| \widehat{E}_b(x) + \int_0^1 \widehat{K}_b(x + tv)v\, dt - E_b(x + v) \right\| \le \varepsilon_E + \varepsilon_K \|v\|.$$

Proof. Subtract Equation 3, apply the triangle inequality, and bound the integral by the operator-norm error times $\|v\|$. □

Quadrature error adds to this bound. An inaccurate predecessor displacement also contributes its propagated error. Finite held-out mean error does not establish the uniform premise.

Proposition 2 (Robust interval margin)**.** *If $|q_i - \widehat{q}_i| \le \varepsilon_i$ and $\ell_i + \varepsilon_i \le \widehat{q}_i \le u_i - \varepsilon_i$ for every monitored coordinate, then $\ell_i \le q_i \le u_i$.*

Proof. Combine the prediction margin and the coordinatewise error bound. □

The implementation uses predicted intervals to rank candidates and actual native intervals to determine success. It does not treat an empirical error statistic as a certified uniform margin.

3.4 When an interface can determine a construction choice

The composition identity makes the interface's role explicit:

$$q(F_B(F_A(x))) - q(x) = E_A(x) + E_B(x + f_A(x)). \tag{5}$$

The second term is precisely the effect whose change is described by Equation 3. It is evaluated at the state produced by A.

Suppose the first displacement is approximated by $\widehat{v}$, and predict the composed effect as

$$\widehat{\Delta}_{AB} = \widehat{E}_A(x) + \widehat{E}_B(x) + \int_0^1 \widehat{K}_B(x + t\widehat{v})\widehat{v}\, dt.$$

If the two present-effect errors are at most $\varepsilon_A, \varepsilon_B$, the context operator error is at most ε_K along the predicted path, and E_B is L_B-Lipschitz between $x + \widehat{v}$ and $x + f_A(x)$, then

$$\|\widehat{\Delta}_{AB} - [q(F_B(F_A(x))) - q(x)]\| \le \varepsilon_A + \varepsilon_B + \varepsilon_K\|\widehat{v}\| + L_B\|\widehat{v} - f_A(x)\|. \tag{6}$$

To obtain the bound, first apply the accumulated-conditioning bound along $\widehat{v}$, then bound the change from its endpoint to the true endpoint. Numerical quadrature contributes its own error. This separates immediate effect fidelity, context fidelity and predecessor-amplitude fidelity.

Proposition 3 (Compatibility decision stability)**.** *Fix a nonempty finite set of physically admissible continuations Π and a nonempty closed interval contract C. Let $d_C(q) = \inf_{c \in C} \|q - c\|_2$ and suppose $\|\widehat{q}_\pi - q_\pi\|_2 \le \varepsilon$ for every $\pi \in \Pi$. If $\widehat{\pi}$ minimizes $d_C(\widehat{q}_\pi)$ and π^* minimizes $d_C(q_\pi)$, then*

$$d_C(q_{\widehat{\pi}}) \le d_C(q_{\pi^*}) + 2\varepsilon.$$

If the true best continuation is unique and every alternative has a distance gap greater than 2ε, the predicted choice equals the true best.

Proof. Distance to a nonempty closed set is 1-Lipschitz. Thus $d_C(q_{\widehat{\pi}}) \le d_C(\widehat{q}_{\widehat{\pi}}) + \varepsilon \le d_C(\widehat{q}_{\pi^*}) + \varepsilon \le d_C(q_{\pi^*}) + 2\varepsilon$. A different selected continuation would contradict the stated gap. □

The search uses squared interval distance, which has the same minimizer. This is a local choice guarantee on a fixed admissible set; native branching checks physical feasibility. It suggests a decision-relevant acquisition target: prediction accuracy relative to the separation between competing continuations. Average held-out derivative error alone does not provide that information.

4 Native substrate and reachable coordinates

We use the public four-bit MLX checkpoint of Qwen3-0.6B [13, 14, 23]. A physical atom is an original SwiGLU neuron with its original normalization, gate/up scalars and down-projection column:

$$a_b(x) = \mathrm{SiLU}((G_l N_l(x))_j)(U_l N_l(x))_j, \qquad f_b(x) = D_l e_j \, a_b(x).$$

These are actual native parameter-backed operations [17, 24]. No fitted weights replace their execution. The loader freezes the model and guards whole-model, transformer-block, attention, embedding and unembedding calls. Original layer order is not a binding rule.

The fixed inventory has 22 physical units inherited from all producer identities proposed in an earlier native pilot, including failures. Its historical discovery work is charged. Native basis probes measure the output directions, forming a full-column-rank matrix $D \in \mathbb{R}^{1024\times 22}$.

Proposition 4 (Reachable-coordinate closure)**.** *If every update has the form $De_b a_b(x)$, the bth column of D times $a_b(x)$, then executions beginning at $x_0 + Dz_0$ remain in $x_0 + \mathrm{im}\, D$. In coordinates, $z \mapsto z + e_b a_b(x_0 + Dz)$, and $D_z E_b(x_0 + Dz) = K_b(x_0 + Dz)D$.*

Proof. Each update adds a column of D times a scalar, so closure follows by induction. The derivative statement is the chain rule. □

Float32 writes introduce small arithmetic residuals; native validation checks actual arrays. We compute current coordinates with $P = (D^\top D)^{-1} D^\top$. This is a supplied physical representation, not a learned semantic vocabulary.

5 Acquisition and intentional composition

What is acquired and what is reused. For each supplied domain Ω, component b retains functions $\widehat{a}_b(z)$ and $\widehat{E}_b(z)$, with context partial $D_z\widehat{E}_b(z)$. These functions form its behavioral interface: the amplitude determines a reachable displacement; the effect predicts a monitored change; the context partial predicts how preceding computation changes that effect. The same functions serve different target/protection masks and demands. The later contract selects compatible continuations without refitting the source model. Acquisition currently samples a supplied domain before those contracts; the experiment does not learn the domain or the physical atom syntax.

The interface forecasts behavior, while the original component executes it. An inaccurate forecast can therefore waste the search allowance, but final success still requires a native execution witness. This division makes it possible to test a representation's construction value independently of native fidelity, and to charge source acquisition separately from repeated use.

5.1 Generic acquired interfaces

For each domain and component we observe its amplitude and accumulated readout effect, $y_b(z) = (a_b(x_0 + Dz), E_b(x_0 + Dz))$. Native source derivatives supply both $\nabla_z a_b$ and $K_b D$. The retained family is a generic quadratic:

$$\widehat{y}_{b,r}(z) = c_{b,r} + L_{b,r} z + \tfrac{1}{2} z^\top H_{b,r} z, \qquad H_{b,r} = H_{b,r}^\top. \tag{7}$$

It has 276 coefficients per scalar field. There are 22 amplitude fields and 22×12 effect fields in each domain. Inputs are scaled by the declared domain radius, outputs by source standard deviation with a fixed floor.

We fit ridge regression with source values and, for the differential arm, their native coordinate derivatives. Value and derivative losses are averaged over source contexts and derivative coordinates; both weights are one, with ridge 10^{-6}. Endpoint controls use the identical family and value loss. No feature or regularization selection uses later results.

The polynomial's derivative and its directional integral are consistent:

$$\widehat{y}_b(z + te_a) - \widehat{y}_b(z) = t\,\partial_a \widehat{y}_b(z) + \tfrac{1}{2}t^2 (H_b)_{aa}.$$

Consistency describes the retained approximation; source and held-out native observations determine its fidelity to the component.

5.2 Structure-calibrated reference

We also supply a reference with the known RMSNorm/SwiGLU form. A normalized linear scalar $g(x) = Ax/r(x)$ satisfies $A = rJ_g + gx^\top/(d\,r)$ at a calibration anchor. Native partials therefore recover the needed row restricted to the reachable directions. For $x = x_0 + Dz$,

$$r(z)^2 = \|x_0\|^2/d + 2x_0^\top Dz/d + z^\top D^\top Dz/d + \varepsilon.$$

Gate, up and readout numerators are affine in z. Original SwiGLU scalars then define component amplitudes and composed effects in reduced coordinates. Calibration costs 113 linear-projection equivalents per domain. This reference receives architectural structure that the generic quadratic learner does not; its source cost and accuracy are reported rather than assumed.

5.3 Compatibility forecasting and native search

For candidate A, let $t = \widehat{a}_A(z)$. The engine predicts A's effect and integrates each successor's context sensitivity along te_A:

$$\widehat{E}_{B|A} = \widehat{E}_B(z) + t\,\partial_A \widehat{E}_B(z) + \tfrac{1}{2}t^2 (H_B)_{AA}.$$

It forecasts the composed readout from $q + \widehat{E}_A + \widehat{E}_{B|A}$. The best compatible successor defines a context/requirement-dependent class for A. Classes therefore arise from predicted composition, rather than fixed semantic names or round-robin cluster quotas.

The common beam engine retains alternative partial native graphs. It permits temporary loss increases and protected-coordinate excursions, while enforcing the physical activity, write, state and domain limits on every actual branch. A native checker determines final interval satisfaction. At each node the ranking can stop after a predicted one-step completion or forecast a successor. Only one-step guidance is used at the final depth or insufficient remaining allowance. Successful or best selected graphs are independently deployed from their original input with native activities recomputed.

6 Central experimental design

Domains and requirements. Three fixed anchors have RMS amplitudes 0.12, 0.25 and 0.6, with consumer layers 14, 20 and 25. Each uses 12 native gate coordinates sampled from anchor values whose absolute magnitude lies in $[0.05, 2]$. Source latent coordinates are uniform in $[-1.5, 1.5]^{22}$; later initial coordinates are uniform in $[-0.5, 0.5]^{22}$. Later components read live state.

Algorithm 1 Acquire, classify, construct, and deploy

```
Measure physical ports and define the compatible reachable domain
Observe source native amplitudes, accumulated effects, and context partials
Fit and retain scoped interfaces; seal their bytes before later requirements
for each later requirement do
    Initialize the native activation frame and bounded search frontier
    while frontier, depth, and query allowance permit do
        Forecast predecessor-conditioned successor effects from retained interfaces
        Classify compatible continuations; rank completion or continuation
        Execute proposed native components and check actual physical limits
        Update the frontier with observed states; recognize native final success
    end while
    Deploy the selected graph; release temporary search states
end for
```

For each of 288 fresh contexts, six readout roles are sampled: three must increase by a fixed cohort demand, and three must remain within ± 0.05 of their initial values. Four ordered cohorts use increases 0.025, 0.05, 0.075 and 0.1. Each cohort contains 24 fresh contexts per domain. The checker allows 10^{-5} numerical violation. Native amplitude is bounded by 2, write RMS by 0.15, live RMS by 2, and latent coordinates by the domain.

Source and controls. Differential acquisition observes 32 contexts per domain. The equal-context endpoint control sees those same value observations. A query-matched endpoint control sees 347 contexts: 22,555 versus 22,560 source LPE per domain. Eight additional contexts per domain diagnose fidelity without model selection. All source objects are sealed before later outcomes.

Nine arms share the same six-depth, four-wide, six-child search engine. They use native local sensitivity, equal-context endpoints, query-matched endpoints, differential paired forecasts, differential one-step forecasts, conditioning removed, conditioning permuted, restored conditioning, or the structured reference. Removing conditioning preserves present effects. Permutation changes the incoming direction associated with each conditioning operator; restoration returns the exact original interface.

Protocol revision and chronology. A first registered batch, P46, completed 288 cases. Code review identified that its paired forecast lacked an explicit terminal option and used two-step ranking at the final depth. The correction was applied symmetrically to all paired arms. P47, the reported final comparison, uses 288 fresh latent contexts and masks with every source model byte unchanged. P46 remains in the record as development; its outcomes are not pooled into the final rate.

Resource envelope. Each search has 512 LPE: a forward native linear projection counts one; the corresponding input-adjoint projection counts one. A native producer gradient has two reverse projections. Norm and scalar work are recorded separately. LPE is an explicit native-query proxy, not a hardware FLOP or energy unit. Scalar planning, fitting, logging and allocation appear in active wall measurements. The retained interface/coordinate payload is bounded by 8 MiB and the producer-activity cache by 1 MiB; full process memory is reported separately.

The lifecycle ledger includes historical inventory discovery, new physical ports and domain setup,

Method	Solved	Source LPE	Lifecycle LPE	Active s
Native local	80	0	130,416	26.55
Endpoint, 32 contexts	100	6,240	109,865	27.94
Endpoint, query-matched	105	67,665	173,739	39.77
Differential interfaces	104	67,680	173,016	34.54
Differential, one-step	102	67,680	173,487	33.95
Conditioning removed	105	67,680	172,401	34.18
Conditioning permuted	108	67,680	172,497	34.26
Restored	104	67,680	173,016	34.50
Structured reference	129	339	102,519	27.38

Table 2: Final comparison. Source LPE excludes common inventory/port work; lifecycle LPE includes it, source acquisition, search and all selected-graph deployment. Active wall includes measured source fitting and CPU planning. All rows use the same online envelope and native quality checker.

required source acquisition and fitting, all search queries, and selected-graph deployment. Validation and independent scientific replay are separately itemized research expenses. The 32-context endpoint source wall cost is measured by a separate exact source-only replay, rather than inferred by proportionally dividing the larger source time. All solver arm orders rotate deterministically across cases.

7 Results

7.1 Construction frontier and lifecycle cost

Table 2 reports the complete final comparison. The learned differential interface satisfies 104 of 288 conditions, compared with 80 for native local guidance and 100 for equal-context endpoint acquisition. The query-matched endpoint control satisfies 105.

The structured reference is strongest: 129 conditions versus 80 for native local guidance, with 21.39% less lifecycle query work. The 32-context endpoint pipeline also increases solve count while reducing this work. These gains concern interface-guided construction under the declared query metric. Their active-wall advantage differs: source acquisition, Python planning, and native kernels have different costs.

At the common 102,519-LPE stream budget, native local guidance has satisfied 73 conditions, 32-context endpoint has 96, the differential pipeline has 56, and the structured reference has 129. The differential acquisition overhead has not amortized against the local baseline under that query budget. At the common 26.55-second active-wall budget, the respective counts are 80, 96, 87, and 124. The source-and-execution ledger is therefore essential to interpreting online query gains.

7.2 Context sensitivity and its causal use

Differential supervision improves held-out K fidelity in each domain, including against the query-matched endpoint learner. Its total-effect prediction does not dominate the query-matched control (Table 3). Approximation quality also varies sharply with domain scale.

Conditioning removal changes 45 selected graphs and satisfies 105 conditions; permutation changes 45 graphs and satisfies 108. Against removal, full conditioning wins one case and loses two; against permutation it wins one and loses five. Restoring the interface reproduces all 288 graphs

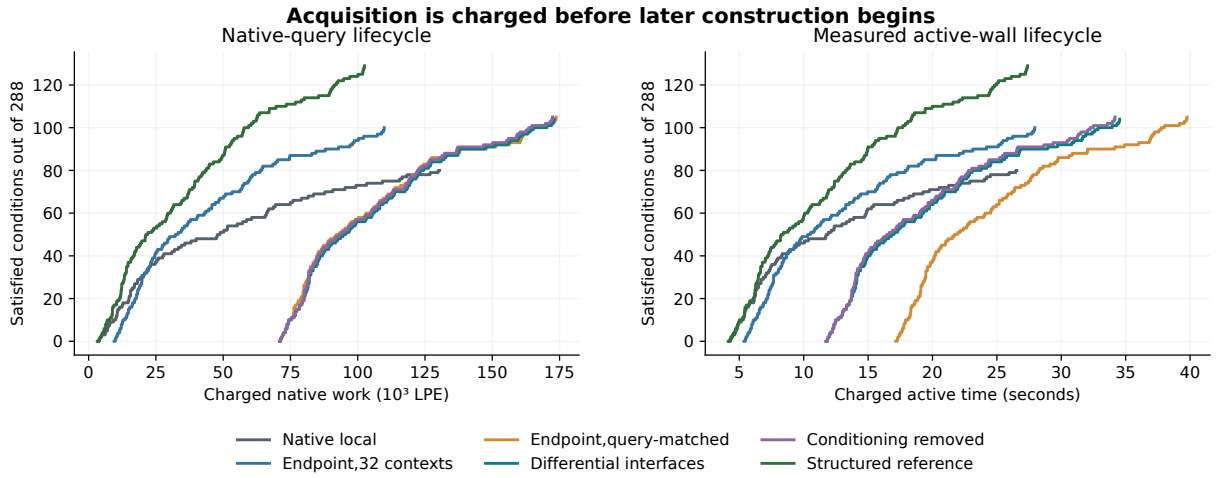


Figure 2: Cumulative fixed-stream quality with acquisition charged at the origin. The two axes expose different resource tradeoffs. Unattempted conditions count as unsolved under an interrupted stream. These curves do not replace paired per-condition outcomes or imply that LPE equals wall time.

Accumulated effect E: RMSE				
Domain	Differential	Endpoint32	Query-matched	Structured
1	0.3573	0.3605	0.3359	4.727e-07
2	0.1801	0.2486	0.162	5.653e-07
3	0.006823	0.04174	0.006529	7.392e-07
Context partial KD: RMSE and native scale				
Domain	Differential	Endpoint32	Query-matched	Native RMS
1	0.1164	0.1242	0.1416	0.129
2	0.05316	0.06347	0.06057	0.06796
3	0.002309	0.01196	0.002697	0.01451

Table 3: Held-out effect and reachable-coordinate context-partial RMSE on eight native contexts per domain, each with 22 components and 12 readouts. The lower panel's final column gives the native partial's RMS scale. Coordinates and bindings sharing a context are correlated; they are not independent task trials.

and final states exactly. Thus conditioning participates in construction, but its incremental solve-count benefit is absent in this realization. Against one-step use of the same differential interface, paired use wins six cases and loses four.

The data distinguish three contributions that should not be conflated: reusable source-derived behavior, multi-step compatibility forecasting, and the learned conditioning increment. Their costs and ablations differ.

7.3 Native fidelity and validation

All 2592 selected programs were independently replayed in a fresh process through the original full hidden activity and masked native down-projection. Full final activation/readout arrays and interval decisions match bit for bit. A deterministic reservoir of 512 branches checks parent reconstruction, activity, physical legality, write and readout. All original parameter bytes remain unchanged.

Method	Domain 1	Domain 2	Domain 3	Total
Native local	15	36	29	80
Endpoint,32 contexts	4	50	46	100
Endpoint,query-matched	7	50	48	105
Differential interfaces	5	51	48	104
Differential,one-step	6	48	48	102
Conditioning removed	7	50	48	105
Conditioning permuted	9	51	48	108
Structured reference	28	53	48	129

Table 4: Satisfied conditions by domain, 96 per column. Heterogeneity matters: a global quadratic is substantially less faithful in the highest relative perturbation domain.

Twenty-four source directional derivatives were additionally checked against direct autodiff of the complete native component/readout program. An initial central-difference step of 0.005 passed 17 of 24 prescribed tolerance checks. The preserved step sweep passes 24 of 24 at 0.02 and worsens as the step shrinks to 0.001 and 0.0002, consistent with cancellation of small effect differences in float32. Direct program autodiff passes all 24 directions. Both the initial failures and the convergence data remain in the evidence record.

The final evaluation process peaks at approximately 530 MiB RSS, including the full frozen checkpoint. Retained quadratic interface/coordinate arrays occupy 3,852,784 bytes across three domains; the structured reference occupies 415,840 bytes. These are logical array payloads, not complete Python or training-workspace memory bounds.

Standalone execution of the acquired bank. We replayed every selected program using the structured representation in a separate NumPy-only process that neither imports MLX nor opens the checkpoint. It recomputes every live component amplitude in reachable coordinates. All 2,592 contract decisions agree with the recorded native decisions; maximum absolute readout error is 3.3021×10^{-6}. The structured formulas occupy 43,104 array bytes across three domains, or 415,840 bytes including output directions, input coordinate projection and anchors. This payload represents the finite 22-unit bank and its fixed monitored readouts. The replay process peaks at 61,702,144 bytes RSS. Its 0.822-second replay measurement includes reading the case-root arrays; it is not a matched native-kernel speedup comparison. This checks that the interface can be executable computational material, rather than only a ranking annotation.

8 Interpretation and scope

The central result is that effect-interface acquisition can make frozen native components usable in a different compositional execution regime, with explicit resource and fidelity checks. The structured reference demonstrates the value of supplying an accurate behavioral representation. Generic endpoint learning also improves the measured construction frontier relative to online local sensitivity.

The contribution connects reusable description to executable composition. Related methods establish layer programs, extracted modules and neural control; our runtime retains a model of what native components do in context and queries it under each new contract. The source object is reused across requirements, while construction can select a different native program each time. The standalone finite-bank replay further shows that an accurate acquired representation can carry executable behavior as well as guide original-native construction.

Derivative-supervised acquisition supplies more accurate context sensitivities, but the present conditioning forecasts do not improve net construction over their removal or permutation. The error bound explains why average derivative fidelity is insufficient: planning needs accurate effects and conditioning along the relevant displacement, at margins small enough for the requirement. Equation 6 identifies the prediction terms that matter, and the decision-stability proposition ties their combined error to the separation between continuations. This identifies the next acquisition target: context-scoped interfaces whose task-relevant effects and conditioning are reliable where compatibility decisions change.

Three design extensions follow directly. Acquisition can concentrate on native construction blockers rather than uniform domain coverage. A local or adaptive representation can replace a single global quadratic where curvature is large. Regression control can learn when an acquired operator is applicable rather than applying it indiscriminately. Each extension should retain the same execution engine and full cost comparisons.

The mathematical formulation applies to compatible differentiable component maps; the empirical study covers one checkpoint, a finite unit bank and supplied numerical readouts. Pretraining-specific advantage over random weights is not isolated. Meaningful semantic contracts, larger inventories, attention fragments, cross-model transfer and repeated acquisition cycles are further evaluations. The present reuse stream measures amortization opportunity, rather than continual compounding across interface revisions.

Lifecycle criterion. For stationary costs, acquisition C_A amortizes against old per-use cost C_0 only when $C_A + n(C_1 + C_M) < nC_0$, where C_1 is new execution and C_M maintenance. If $C_0 > C_1 + C_M$, break-even requires $n > C_A/(C_0 - C_1 - C_M)$. The study instead reports cumulative measured curves, because demands and success vary. Acquisition, storage, prediction and maintenance are part of the method's resource claim.

9 Research program

The present paper establishes a common object, runtime and measurement protocol for studying acquired access to frozen computation. The next stages ask how that access can become more useful, accumulate under a budget, and support capabilities with external meaning. Each stage should keep the source knowledge, construction procedure and lifecycle accounting explicit.

Decision-relevant acquisition. Which native observations should the system acquire next? The composition-error bound separates effect, context and displacement errors; compatibility stability depends on those errors relative to the separation between continuations. This suggests acquiring distinctions where a construction decision is unresolved, with local applicability and calibrated uncertainty. The empirical test is whether targeted acquisition improves later native construction enough to repay its additional observations, against fixed-data and equal-work controls.

Continual acquisition and consolidation. Can a sequence of solving, acquiring descriptions, consolidating and reusing them reduce the cost of acquiring subsequent capabilities? This requires repeated interface revisions, not just reuse of one sealed source fit. Comparisons should include fixed and extract-once repertoires under matched retained-state and active-workspace limits. Failed acquisitions, consolidation, eviction, rediscovery and regression checks belong in the same cost

ledger. The target is a sustained improvement in later cost-to-quality as demand rises, measured against those controls over a declared horizon.

Broader capability and resource scaling. How can internally recognizable contracts be grounded in externally useful capabilities? This stage connects native construction to externally checked tasks, expands component banks and readouts, and repeats the method across pretrained architectures. Random or otherwise matched substrate controls can isolate the contribution of pretraining. Active footprint, retained state, total computation and latency should be measured separately as demand grows. These experiments would test how far improved organization can substitute for additional resources in a concrete application.

The long-term objective is a system whose accumulated descriptions of frozen computation support progressively more capable executable organizations within a bounded active envelope. Shaping Gap Benchmark supplies a starting point: explicit native contracts, acquired behavioral interfaces, executable witnesses and a shared acquisition-inclusive criterion for judging each subsequent step.

10 Conclusion

We introduced Shaping Gap Benchmark and a runtime that acquires reusable behavioral interfaces over ordinary frozen pretrained components and uses them to construct native programs satisfying explicit neural requirements. Accumulated effects describe what a component changes; incoming-context partials describe how predecessors change that effect. Together they give the runtime an operational description for composition, with conditional error and decision-stability guarantees.

The study connects this representation to native witnesses and acquisition- inclusive lifecycle curves. Endpoint interfaces improve the measured construction frontier; structure calibration gives the strongest result. Derivative supervision improves context prediction, while the present conditioning policy adds no net solve-count benefit. These results make the next mechanism concrete: acquire behavioral knowledge with sufficient decision-relevant precision to make later constructions reliably cheaper.

References

[1] Yamini Bansal, Preetum Nakkiran, and Boaz Barak. Revisiting model stitching to compare neural representations, 2021. URL https://arxiv.org/abs/2106.07682.

[2] Arthur Conmy, Augustine N. Mavor-Parker, Aengus Lynch, Stefan Heimersheim, and Adrià Garriga-Alonso. Towards automated circuit discovery for mechanistic interpretability. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://arxiv.org/abs/2304.14997.

[3] Wojciech M. Czarnecki, Simon Osindero, Max Jaderberg, Grzegorz Swirszcz, and Razvan Pascanu. Sobolev training for neural networks. In *Advances in Neural Information Processing Systems*, 2017. URL https://papers.nips.cc/paper_files/paper/2017/hash/758a06618c69880a6cee5314ee42d52f-Abstract.html.

[4] Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sable-Meyer, Luc Cary, Lucas Morales, Luke Hewitt, Armando Solar-Lezama, and Joshua B. Tenenbaum. Dreamcoder: Growing

generalizable, interpretable knowledge with wake-sleep Bayesian program learning, 2020. URL https://arxiv.org/abs/2006.08381.

[5] Marco Fumero, Marco Pegoraro, Valentino Maiorca, Francesco Locatello, and Emanuele Rodolà. Latent functional maps: A spectral framework for representation alignment. In *Advances in Neural Information Processing Systems*, volume 37, pages 66178–66203, 2024. URL https://arxiv.org/abs/2406.14183.

[6] Gabriel Grand, Lionel Wong, Maddy Bowers, Theo X. Olausson, Muxin Liu, Joshua B. Tenenbaum, and Jacob Andreas. LILO: Learning interpretable libraries by compressing and documenting code. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2310.19791.

[7] Joseph D. Janizek, Pascal Sturmfels, and Su-In Lee. Explaining explanations: Axiomatic feature interactions for deep networks. *Journal of Machine Learning Research*, 22(104):1–54, 2021. URL https://jmlr.csail.mit.edu/beta/papers/v22/20-1223.html.

[8] Hiroaki Kingetsu, Kenichi Kobayashi, and Taiji Suzuki. Neural network module decomposition and recomposition, 2021. URL https://arxiv.org/abs/2112.13208.

[9] George Konidaris, Leslie Pack Kaelbling, and Tomás Lozano-Pérez. From skills to symbols: Learning symbolic representations for abstract high-level planning. *Journal of Artificial Intelligence Research*, 61:215–289, 2018. URL https://cs.brown.edu/people/gdk/pubs/orig_sym_jair.pdf.

[10] Ziyue Li, Yang Li, and Tianyi Zhou. Skip a layer or loop it? test-time depth adaptation of pretrained LLMs, 2025. URL https://arxiv.org/abs/2507.07996.

[11] Ziyue Li, Yang Li, and Tianyi Zhou. Skip a layer or loop it? learning program-of-layers in LLMs, 2026. URL https://arxiv.org/abs/2606.06574. Version 2, August 2026.

[12] Zichang Liu, Jue Wang, Tri Dao, Tianyi Zhou, Binhang Yuan, Zhao Song, Anshumali Shrivastava, Ce Zhang, Yuandong Tian, Christopher Re, and Beidi Chen. Deja vu: Contextual sparsity for efficient LLMs at inference time. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 22137–22176, 2023. URL https://proceedings.mlr.press/v202/liu23am.html.

[13] MLX LM Contributors. MLX LM, 2026. URL https://github.com/ml-explore/mlx-lm. Version 0.32.2; experiments use MLX 0.32.4.

[14] Qwen Team. Qwen3-0.6B-MLX-4bit, 2025. URL https://huggingface.co/Qwen/Qwen3-0.6B-MLX-4bit. Checkpoint revision 173234aa840d113125e9f2271100ddbaf16c9620.

[15] Nasim Rahaman, Muhammad Waleed Gondal, Shruti Joshi, Peter Gehler, Yoshua Bengio, Francesco Locatello, and Bernhard Schölkopf. Dynamic inference with neural interpreters. In *Advances in Neural Information Processing Systems*, 2021. URL https://arxiv.org/abs/2110.06399.

[16] Clemens Rosenbaum, Tim Klinger, and Matthew Riemer. Routing networks: Adaptive selection of non-linear functions for multi-task learning, 2017. URL https://arxiv.org/abs/1711.01239.

[17] Noam Shazeer. GLU variants improve transformer, 2020. URL https://arxiv.org/abs/2002.05202.

[18] Tom Silver, Ashay Athalye, Joshua B. Tenenbaum, Tomás Lozano-Pérez, and Leslie Pack Kaelbling. Learning neuro-symbolic skills for bilevel planning. In *Proceedings of the 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 701–714, 2023. URL https://proceedings.mlr.press/v205/silver23a.html.

[19] Julian Skifstad, Xinyue Annie Yang, and Glen Chou. Local linearity of LLMs enables activation steering via model-based linear optimal control, 2026. URL https://arxiv.org/abs/2604.19018.

[20] Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic attribution for deep networks. In *Proceedings of the International Conference on Machine Learning*, pages 3319–3328, 2017. URL https://proceedings.mlr.press/v70/sundararajan17a.html.

[21] Lazar Valkov, Dipak Chaudhari, Akash Srivastava, Charles Sutton, and Swarat Chaudhuri. HOUDINI: Lifelong learning as program synthesis. In *Advances in Neural Information Processing Systems*, volume 31, 2018. URL https://arxiv.org/abs/1804.00218.

[22] Martin Weiss, Nasim Rahaman, Francesco Locatello, Chris Pal, Yoshua Bengio, Bernhard Schölkopf, Li Erran Li, and Nicolas Ballas. Neural attentive circuits. In *Advances in Neural Information Processing Systems*, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/32f227c41a0b4e36f65bebb4aeda94a2-Abstract-Conference.html.

[23] An Yang et al. Qwen3 technical report, 2025. URL https://arxiv.org/abs/2505.09388.

[24] Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL https://arxiv.org/abs/1910.07467.

A Artifact and experimental chronology

The physical inventory was fixed before the present source acquisition. It contains all 22 producer identities proposed by an earlier native calibration, including failed proposals. Its historical discovery work is charged to every method. The artifact records both the inventory and its inherited cost receipt.

The source-acquisition batch (P46) measures native directions, samples the three source domains, fits the interface controls, calibrates the structured reference and seals their arrays. Its eight validation contexts per domain are diagnostic. The subsequent development constructions exposed the missing terminal option in paired forecasts. The same stop-aware correction was then applied to all paired controls before the final batch (P47), which uses 288 fresh states and masks. The final study reuses every sealed source-model byte and does not pool the development outcomes into its comparison.

The supporting artifact links protocols, source arrays, selected programs and cost receipts to the reported tables. It also preserves the original numerical verification history: the failed small-step finite-difference check, the step sweep and the final direct-autodiff checks.

B Recorded reproducibility boundary

The checkpoint revision is `173234aa840d113125e9f2271100ddbaf16c9620`. The loaded stored-parameter SHA-256 is

`27a04e3e5e6b1d524751fa34d1053044bce9`
`ce7e4995e5b63b5c6b2a2dfe3798`

The recorded environment uses NumPy 2.5.4, MLX 0.32.4 and MLX-LM 0.32.2 on the local Apple Metal runtime. Parameter bytes total 316,747,776. Native bitwise replay is a statement about this recorded runtime; it does not assert cross-device determinism.

The evidence package retains registered local protocols, exact runner/helper snapshots, source arrays and interface coefficients, all case roots, complete primitive/branch records, failed verification receipts, source derivative convergence, final native verification, cost ledgers and evidence-derived tables. Full checkpoint files are excluded; acquired interfaces and derived native observations are included. The companion repository URL is https://github.com/calysteon/shaping-gap-benchmark. It includes the paper and its source, portable experiment entry points, pinned source observations, contracts, selected programs and result receipts. Fresh native runs regenerate complete primitive and branch traces.

B.1 Inventory and prospective seeds

The physical inventory is recorded below. IDs are original layer/neuron coordinates, not authored operation labels.

Layer	Neuron indices
6	2069
10	211
13	573
16	2273
18	74, 804, 1234, 1741, 1839, 2230, 2465, 2490, 2599
19	622, 2864
20	2602
21	2259, 2386
23	1226
24	1914, 2701

Anchor seeds are 61001–61003, readout-selection seeds 61501–61503, source seeds 62001–62003 and validation seeds 63001–63003. The fixed endpoint budget selects the first 347 source draws; the differential arm uses the first 32. P46 uses later seeds 64001–64288 and P47 uses 66001–66288. Requirement masks use each later seed plus 900000. The direction-permutation control uses seed 65001. NumPy's recorded default generator produces all draws.

The host is an Apple M4 Max with 36 GiB physical memory. Timing values are single recorded runs with deterministic arm-order rotation; the study reports paired outcomes and complete query counts alongside those measurements. The compact reproduction package contains acquired arrays, code, protocols and case roots. Full primitive/branch traces are retained separately, with content hashes connecting them to the results.